

How Nginx Was Able to Support 1 Million Concurrent Connections on a Single Server ✨

#62: Break Into Nginx Architecture (4 Minutes)



NEO KIM

DEC 04, 2024



Get my system design playbook for FREE on newsletter signup:

✓ Subscribed

This post outlines Nginx architecture. You will find references at the bottom of this page if you want to go deeper.

- [Share this post](#) & I'll send you some rewards for the referrals.

Note: This post is based on my research and may differ from real-world implementation.

Once upon a time, there was a tech startup.

They offered invoice generation services.

Yet they had only a few users.

So they ran their site on a single web server - Apache HTTP.

Life was good.



But one day they started to receive massive traffic.

Yet their server had only limited capacity.

So they set up more servers.

Although it temporarily solved their scalability issues, there were newer problems.

Here are some of them:

1. Resource Usage

A connection to the web server needs a separate thread or process.

Also consumes memory and CPU time until it's released.

So computing resources get wasted.

2. Performance

A thread must wait until a *blocking task* is complete.

Besides context switch increases with many threads.

Think of the context switch as switching a CPU between different threads.

So performance is bad.

Onward.

Eraser - Sponsor



Eraser is an AI co-pilot for technical design that can help your team deliver accurate, consistent designs faster. Create beautiful technical diagrams in seconds from natural language prompts or code prompts (Terraform or SQL).

[Sign up for free](#)

How Does Nginx Work

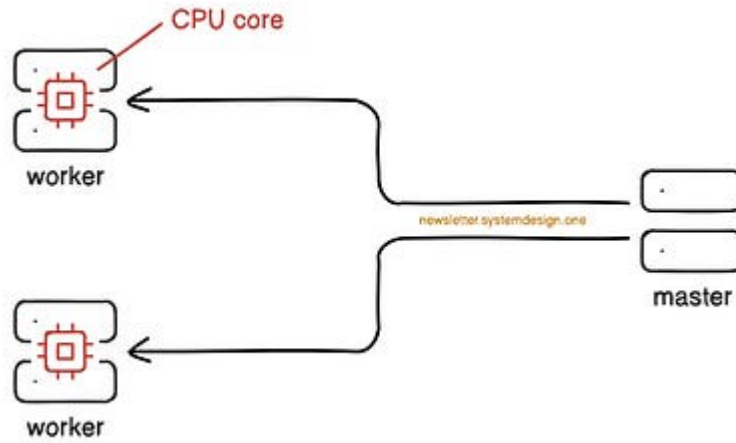
They wanted to reduce costs and ditch the complexity of many servers.

So they moved to Nginx.

And here's how it offers extreme scalability:

1. Parallelism

Running many tasks at the same time is called **parallelism**.



Master-Worker Model in Nginx

They built Nginx with the master-worker model:

- The master reads the configuration file and creates new workers.
- While the worker handles client connections.

A simple approach is to give each connection a separate thread or process.

Yet it will result in context switches, thread trashing, and memory usage.

Imagine **thread trashing** as simply switching between threads instead of doing actual work.

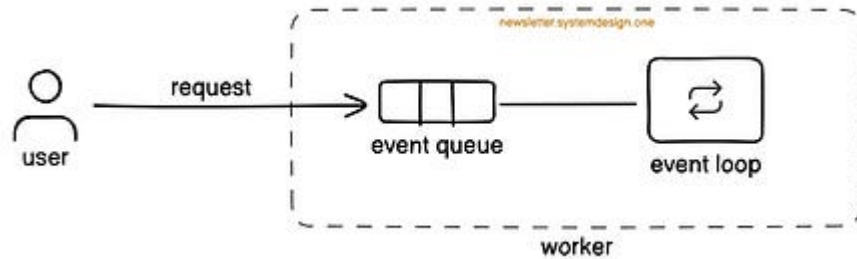
So they run *each worker as a separate process*. And assign a separate CPU to each worker. Put simply, the number of workers equals the number of CPUs.

It gives better performance.

Ready for the best part?

2. Concurrency

Managing many tasks at the same time is called **concurrency**. (Yet all tasks don't have to run at the same time.)



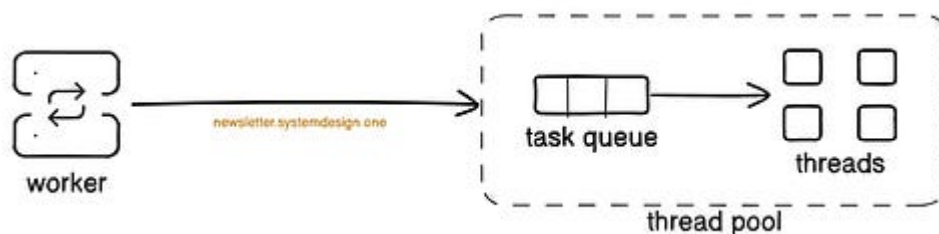
A Worker Processing Requests Using the Event Loop

They run the worker as a single-threaded process:

- The client sends a request to the worker.
- The request gets added to its event queue.
- The worker processes the request using the event loop.

While event loop is asynchronous, event-driven, and non-blocking.

Think of the **event loop** as the chef in a busy kitchen - they prepare many orders by doing 1 task at a time. And switch between orders when ready.

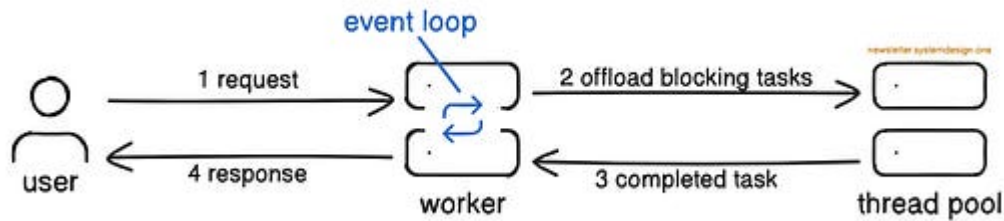


A Worker Delegating a Blocking Task to the Thread Pool

A CPU-intensive task will keep the event loop busy. (Also block the worker from accepting new requests.)

So they *offload blocking tasks to the thread pool*.

Think of the **thread pool** as a group of threads created at Nginx startup. And they're shared between workers.



Workflow of Processing a Blocking Request

Here's how it works:

- A worker delegates a blocking task to the thread pool.
- The worker then handles other requests using the event loop.
- The thread pool tells the worker once the blocking task is complete.
- The worker responds to the client.

This approach keeps the event loop responsive.

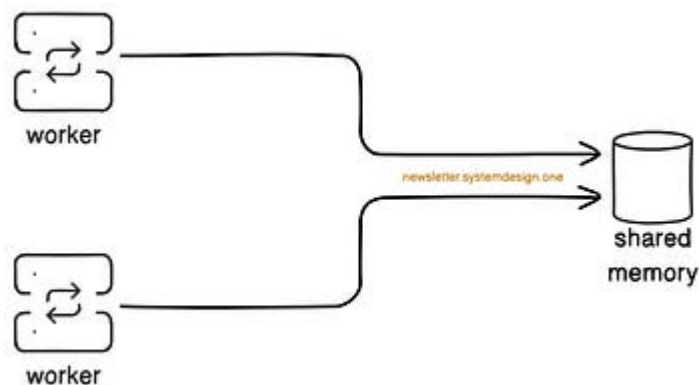
Ready for the next technique?

3. Scalability

A simple way to scale is by caching responses.

Yet it's inefficient to cache the same data across different workers.

So they set up **shared memory**.



Workers Accessing the Same Data via Shared Memory

It let them:

- Share cache data across workers.
- Store session data and route requests to a specific backend server.
- Rate limit by tracking requests.

Besides it reduces the memory usage of Nginx.

While workers use a lock (**mutex**) to access shared memory.



A single Nginx server can handle 1 million concurrent connections.

Yet it isn't a silver bullet to scalability.

So choose the right tool for your needs.

Subscribe to get simplified case studies delivered straight to your inbox:

✓ Subscribed

Neo's Recommendations

- Product for Engineers: Actionable advice to improve your product skills as a software engineer.
- The Developing Dev: Mentorship from a staff engineer at Instagram.



Follow me on [LinkedIn](#) | [YouTube](#) | [Threads](#) | [Twitter](#) |
[Instagram](#)



Want to advertise in this newsletter?

If your company wants to reach a tech audience, you may want to advertise with me.

Thank you for supporting this newsletter. Consider sharing this post with your friends and get rewards. Y'all are the best.

1 Referral



**Leetcode
Master
Template**

2 Referrals



**Interview
Mistakes
to Avoid PDF**

3 Referrals



**Popular
Interview
Questions PDF**

Share



How Databases Keep Passwords Securely 🔒

NEO KIM • NOVEMBER 18, 2024

[Read full story](#)



How Google Ads Was Able to Support 4.77 Billion Users With a SQL Database 🔥

NEO KIM • NOVEMBER 9, 2024

[Read full story](#)



References

- [Inside NGINX: How We Designed for Performance & Scale](#)
- [The Powerful & Efficient NGINX Architecture](#)
- [The Architecture of Open Source Applications \(Volume 2\) - nginx](#)
- [Thread Pools in NGINX Boost Performance 9x!](#)
- [Tuning NGINX for Performance](#)
- [Socket Sharding in NGINX Release](#)
- [Controlling nginx](#)
- [The C10K problem](#)
- [Why does one NGINX worker take all the load?](#)
- [Does Nginx block on file IO?](#)
- Block diagrams created with [Eraser](#)